

Atributos e Métodos Estáticos

Membros que pertencem à classe, não ao objeto

Linguagem de Programação II

Revisão rápida: classe vs. objeto

Cada vez que usamos `new`, criamos um objeto **independente** na memória:

```
ContaBancaria c1 = new ContaBancaria("Maria");  
ContaBancaria c2 = new ContaBancaria("Joao");  
ContaBancaria c3 = new ContaBancaria("Ana");
```

c1

titular: Maria
saldo: 0

c2

titular: João
saldo: 0

c3

titular: Ana
saldo: 0

- Cada objeto tem seu **próprio** titular e saldo.
- Alterar o saldo de c1 não afeta c2 nem c3.

Pergunta

E se quisermos guardar **quantos objetos foram criados no total**? Onde colocar essa informação?

O problema: informação que pertence à classe

Imagine que queremos contar quantas contas bancárias existem.

```
// Tentativa errada: atributo de instancia  
public class ContaBancaria {  
    private String titular;  
    private double saldo;  
    private int totalContas; // Cada objeto teria o SEU contador!  
}
```



Problema

Com atributo de instância, **cada objeto teria seu próprio totalContas**. Criar c1 não incrementaria o contador de c2.



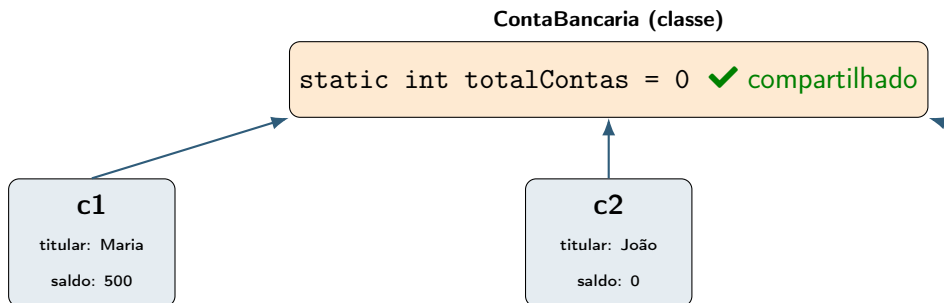
Solução

Precisamos de um atributo que seja **compartilhado por todos os objetos** da classe, ou seja, que pertença à **classe em si** — e não a cada instância.

O modificador static

Definição

Um membro declarado com `static` pertence à **classe**, não a um objeto específico. Existe **uma única cópia** na memória, independentemente de quantos objetos forem criados.



todos enxergam o mesmo totalContas

Atributo estático: contador de objetos

```
public class ContaBancaria {  
  
    private String titular;  
    private double saldo;  
  
    // Atributo estatico: pertence a CLASSE, nao ao objeto  
    private static int totalContas = 0;  
  
    public ContaBancaria(String titular) {  
        this.titular = titular;  
        this.saldo = 0;  
        totalContas++; // incrementa o contador compartilhado  
    }  
  
    // Metodo estatico para consultar o contador  
    public static int getTotalContas() {  
        return totalContas;  
    }  
}
```

- totalContas existe **uma única vez** na memória.
- Todo construtor que roda incrementa **o mesmo** contador.

Usando o atributo estático no Main

```
class Main {  
    public static void main(String[] args) {  
  
        System.out.println(ContaBancaria.getTotalContas()); // 0  
  
        ContaBancaria c1 = new ContaBancaria("Maria");  
        System.out.println(ContaBancaria.getTotalContas()); // 1  
  
        ContaBancaria c2 = new ContaBancaria("Joao");  
        ContaBancaria c3 = new ContaBancaria("Ana");  
        System.out.println(ContaBancaria.getTotalContas()); // 3  
    }  
}
```

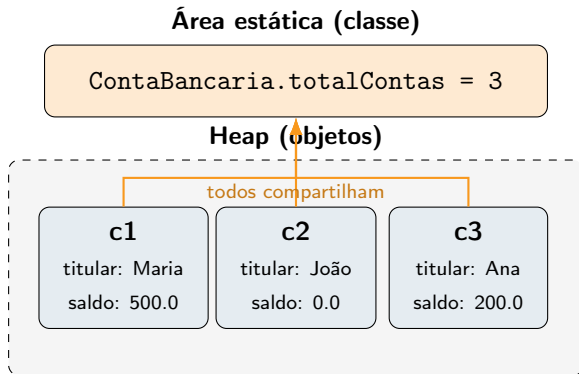
> Saída esperada

0
1
3



Observe

O que acontece na memória?



- Atributos de instância (titular, saldo) ficam **no heap**, um por objeto.
- Atributo estático (totalContas) fica na **área estática**, único para todos.

Métodos estáticos

Definição

Um método `static` pode ser chamado **sem criar um objeto**. Basta usar o nome da classe seguido do método.

✓ Método estático

Não precisa de objeto:

```
Math.sqrt(16)
```

```
Integer.parseInt("42")
```

```
ContaBancaria.getTotalContas()
```

Chama pelo **nome da classe**.

i Método de instância

Precisa de um objeto:

```
conta.depositar(100)
```

```
conta.getSaldo()
```

```
aluno.calcularMedia()
```

Chama pelo **objeto**.

💡 Regra simples

Se o método **não depende** de nenhum dado do objeto (não usa `this`), ele pode ser `static`.

Exemplo real: a classe Math do Java

A classe Math é um exemplo clássico de métodos estáticos:

```
// Nao precisamos criar nenhum objeto Math!  
double raiz    = Math.sqrt(25);      // 5.0  
double potenc  = Math.pow(2, 10);    // 1024.0  
int    maximo  = Math.max(8, 13);    // 13  
double abs     = Math.abs(-7.5);     // 7.5  
double pi      = Math.PI;            // 3.14159...
```

- `Math.sqrt()`, `Math.pow()` etc. são métodos `static`.
- `Math.PI` é um **atributo estático** (além de `final`).
- Não faz sentido criar `new Math()` para calcular uma raiz.

Por que estático faz sentido aqui?

O resultado de `Math.sqrt(25)` não depende de nenhum *estado* de objeto. É sempre 5.0, independentemente de qualquer coisa.

Criando nossa própria classe utilitária

Assim como Math, podemos criar classes com métodos estáticos:

```
public class Calculadora {  
  
    // Metodo estatico: recebe valores, devolve resultado  
    // Nao usa nenhum atributo de objeto  
    public static double somar(double a, double b) {  
        return a + b;  
    }  
  
    public static double media(double[] valores) {  
        double soma = 0;  
        for (double v : valores) {  
            soma += v;  
        }  
        return soma / valores.length;  
    }  
  
    public static boolean ePositivo(double numero) {  
        return numero > 0;  
    }  
}
```

Usando a classe Calculadora

```
class Main {  
    public static void main(String[] args) {  
  
        // Chamando diretamente pelo nome da classe  
        double resultado = Calculadora.somar(10.0, 5.5);  
        System.out.println(resultado); // 15.5  
  
        double[] notas = {7.0, 8.5, 6.0, 9.0};  
        double media = Calculadora.media(notas);  
        System.out.println(media); // 7.625  
  
        boolean positivo = Calculadora.ePositivo(-3.0);  
        System.out.println(positivo); // false  
    }  
}
```

✓ Vantagem

Não precisamos criar um objeto Calculadora para usar os métodos. Eles estão sempre disponíveis pelo nome da classe.

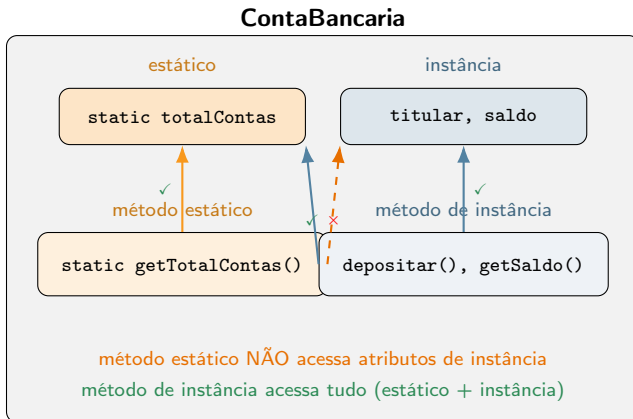
A restrição mais importante do static

⚠ Regra fundamental

Um método static **não pode acessar** atributos ou métodos de instância (não-estáticos) diretamente.

```
public class ContaBancaria {  
    private String titular;           // instancia  
    private double saldo;             // instancia  
    private static int totalContas;   // estatico  
  
    public static void exemploErrado() {  
        // ERRO: nao sabe de QUAL objeto pegar o saldo  
        System.out.println(saldo);    // ERRO DE COMPILACAO!  
        System.out.println(titular);  // ERRO DE COMPILACAO!  
  
        // OK: acessa o atributo estatico  
        System.out.println(totalContas); // correto  
    }  
}
```

O que cada tipo de método pode acessar?



Constantes estáticas: static final

Muitas vezes usamos static junto com final para criar **constantes**:

```
public class ContaBancaria {  
  
    // Constante: nao muda nunca, pertence a classe  
    public static final double LIMITE_SAQUE_DIARIO = 5000.0;  
    public static final int    MAX_TENTATIVAS      = 3;  
  
    private String titular;  
    private double saldo;  
  
    public boolean sacar(double valor) {  
        if (valor > LIMITE_SAQUE_DIARIO) {    // usa a constante  
            return false;  
        }  
        if (valor > 0 && valor <= saldo) {  
            saldo -= valor;  
            return true;  
        }  
        return false;  
    }  
}
```

Acessando constantes static final

```
class Main {  
    public static void main(String[] args) {  
  
        // Acessa a constante pelo nome da classe  
        System.out.println(ContaBancaria.LIMITE_SAQUE_DIARIO);  
        // Saída: 5000.0  
  
        System.out.println(ContaBancaria.MAX_TENTATIVAS);  
        // Saída: 3  
  
        ContaBancaria conta = new ContaBancaria("Maria");  
        conta.depositar(10000);  
  
        // Tenta sacar acima do limite  
        boolean ok = conta.sacar(6000);  
        System.out.println(ok); // false  
    }  
}
```

> Saída esperada

5000.0 3 false

Por que usar constantes `static final`?

✓ Vantagens das constantes

- Valor definido em **um único lugar**.
- Fácil de alterar sem buscar no código todo.
- Nome descritivo: mais legível que `5000.0`.

💡 Convenção

Constantes em Java são escritas em **MAIUSCULAS_COM_UNDERLINE**

- `LIMITE_SAQUE_DIARIO`
- `PI`
- `MAX_VALOR`

⚠ Sem constante vs. com constante

Sem constante

```
if (valor > 5000.0)
```

O que significa 5000?

Repetido em 10 lugares?

Com constante

```
if (valor >  
    LIMITE_SAQUE)
```

Autoexplicativo.

Muda em um lugar só.

Classe ContaBancaria com membros estáticos

```
public class ContaBancaria {  
  
    // Constante estatica  
    public static final double LIMITE_SAQUE = 5000.0;  
  
    // Atributo estatico: compartilhado por todos os objetos  
    private static int totalContas = 0;  
  
    // Atributos de instancia: cada objeto tem o seu  
    private String titular;  
    private double saldo;  
  
    public ContaBancaria(String titular) {  
        this.titular = titular;  
        this.saldo = 0;  
        totalContas++; // incrementa o contador compartilhado  
    }  
  
    public void depositar(double valor) {  
        if (valor > 0) saldo += valor;  
    }  
  
    public boolean sacar(double valor) {  
        if (valor > 0 && valor <= saldo && valor <= LIMITE_SAQUE) {  
            saldo -= valor;  
            return true;  
        }  
        return false;  
    }  
  
    public double getSaldo()          { return saldo; }  
    public String getTitular()        { return titular; }
```

Outro exemplo: classe Aluno com contador

```
public class Aluno {  
  
    private static int totalAlunos = 0; // compartilhado  
  
    private String nome;  
    private double nota1;  
    private double nota2;  
  
    public Aluno(String nome, double nota1, double nota2) {  
        this.nome = nome;  
        setNota1(nota1);  
        setNota2(nota2);  
        totalAlunos++; // cada new Aluno incrementa  
    }  
  
    public void setNota1(double n) { if(n>=0&&n<=10) nota1=n; }  
    public void setNota2(double n) { if(n>=0&&n<=10) nota2=n; }  
  
    public double calcularMedia() { return (nota1 + nota2) / 2; }  
    public boolean estaAprovado() { return calcularMedia() >= 7; }  
  
    // Metodo estatico  
    public static int getTotalAlunos() { return totalAlunos; }  
}
```

Testando a classe Aluno no Main

```
class Main {  
    public static void main(String[] args) {  
  
        System.out.println(Aluno.getTotalAlunos()); // 0  
  
        Aluno a1 = new Aluno("Carlos", 8.0, 7.5);  
        Aluno a2 = new Aluno("Beatriz", 9.0, 8.5);  
        Aluno a3 = new Aluno("Pedro", 5.0, 6.0);  
  
        System.out.println(Aluno.getTotalAlunos()); // 3  
  
        System.out.println(a1.calcularMedia()); // 7.75  
        System.out.println(a1.estaAprovado()); // true  
  
        System.out.println(a3.calcularMedia()); // 5.5  
        System.out.println(a3.estaAprovado()); // false  
    }  
}
```

> Saída

0 3 7.75 true 5.5 false

Comparando: membro de instância vs. estático

	Instância	Estático (static)
Pertence a	Cada objeto	A classe toda
Cópias na memória	Uma por objeto	Uma única
Como acessar	Precisa de objeto	Pelo nome da classe
Usa <code>this</code> ?	Sim	Não
Exemplos	saldo, titular, depositar()	totalContas, Math.sqrt(), PI



Resumo

- Dados que variam **por objeto** → atributo de instância.
- Dados **compartilhados** ou constantes → `static`.
- Métodos que **não dependem** do estado do objeto → `static`.

Quando usar static?

✓ Use static para...

- **Contadores** de objetos criados
- **Constantes** (PI, limites, valores fixos)
- **Funções utilitárias** que não dependem de objeto (como Math)
- **Fábricas:** métodos que criam e retornam objetos

✗ Não use static para...

- Dados que **variam por objeto** (saldo, nome, nota)
- Métodos que **usam** this ou atributos de instância
- Substituição de orientação a objetos (abusar de static leva a código procedural)

⚠ Armadilha comum para iniciantes

Colocar static em **tudo** para “resolver” erros de compilação sem entender o motivo. Isso destrói o conceito de orientação a objetos.

Por que o main é static?

```
public class Main {  
    public static void main(String[] args) {  
        // ponto de entrada do programa  
    }  
}
```

Explicação

A JVM (máquina virtual Java) precisa chamar o main **sem criar nenhum objeto** primeiro. Por isso ele deve ser static: pode ser chamado diretamente pelo nome da classe.

✗ Erro clássico

```
public class Exemplo {  
    int valor = 10;  
  
    public static void main(String[] args)  
    {  
        // ERRO: nao pode acessar  
        // valor sem objeto!  
    }  
}
```

✓ Correto

```
public class Exemplo {  
    int valor = 10;  
  
    public static void main(String[] args)  
    {  
        Exemplo e = new Exemplo();  
        System.out.println(e.valor); // OK  
    }  
}
```

```

                                sem static;
[draw, diamond, aspect=2.8, fill=JavaOrange!20, below right=1.0cm and 1.8cm of q1] (q2) É
                                uma constante; [font=, below=0.05cm of q2] ou valor fixo?;
[draw, rounded corners, fill=JavaOrange!20, below left=1.0cm and 0.5cm of q2, minimum
                                width=3.2cm] (cst) Constante
                                static final;
[draw, rounded corners, fill=JavaGreen!20, below right=1.0cm and 0.5cm of q2, minimum
                                width=2.8cm] (stc) Estático
                                static;
[-Latex, thick] (inicio) – (q1); [-Latex, thick] (q1) – node[above left]Sim (inst); [-Latex, thick]
(q1) – node[above right]Não (q2); [-Latex, thick] (q2) – node[above left]Sim (cst); [-Latex,
                                thick] (q2) – node[above right]Não (stc);

```

Exercício 1: Contador de produtos

Crie uma classe Produto com:

- Atributos privados de instância: String nome, double preco
- Atributo estático privado: int totalProdutos
- Construtor que valida o preço (≥ 0) e incrementa o contador
- setPreco(double) com validação
- getPreco(), getNome()
- static getTotalProdutos()

Teste esperado no Main

```
System.out.println(Produto.getTotalProdutos()); // 0
Produto p1 = new Produto("Caneta", 2.50);
Produto p2 = new Produto("Caderno", 15.90);
System.out.println(Produto.getTotalProdutos()); // 2
p1.setPreco(-1.0); // deve ser ignorado
System.out.println(p1.getPreco()); // 2.50
```


Exercício 2: Classe utilitária ValidadorCPF

Crie uma classe ValidadorCPF com métodos **apenas estáticos**:

- static boolean temTamanhoCorreto(String cpf)
retorna true se o CPF tem exatamente 11 dígitos
- static boolean apenasDigitos(String cpf)
retorna true se todos os caracteres são números
- static boolean eValido(String cpf)
retorna true se o CPF é válido (combina os dois acima)

Teste esperado

```
System.out.println(ValidadorCPF.eValido("12345678901")); // true
System.out.println(ValidadorCPF.eValido("123"));          // false
System.out.println(ValidadorCPF.eValido("1234567890a")); // false
```

Dica

Use `cpf.length()` e `Character.isDigit(c)` para verificar os caracteres.

Exercício 3: Identifique os erros

O código abaixo tem erros relacionados ao uso de static. Identifique e corrija:

```
public class Turma {  
  
    private static String nome;        // erro?  
    private int totalAlunos = 0;      // erro?  
  
    public Turma(String nome) {  
        this.nome = nome;  
    }  
  
    public static void adicionarAluno() {  
        totalAlunos++; // erro?  
    }  
  
    public static int getTotal() {  
        return totalAlunos; // erro?  
    }  
}
```



Perguntas

Solução do Exercício 3

```
public class Turma {  
  
    // nome e estatico: todas as turmas teriam o mesmo nome!  
    // Correto: cada turma tem seu proprio nome  
    private String nome; // instancia  
  
    // totalAlunos deve ser compartilhado entre objetos  
    private static int totalAlunos = 0; // estatico  
  
    public Turma(String nome) {  
        this.nome = nome;  
    }  
  
    // Correto: metodo estatico acessando atributo estatico  
    public static void adicionarAluno() {  
        totalAlunos++;  
    }  
  
    public static int getTotal() {  
        return totalAlunos;  
    }  
}
```

Exercício 4: Classe Estacionamento

Modele a classe Estacionamento com:

Atributos:

- static final int
CAPACIDADE = 50
- static int vagasOcupadas
= 0
- String placa (instância)

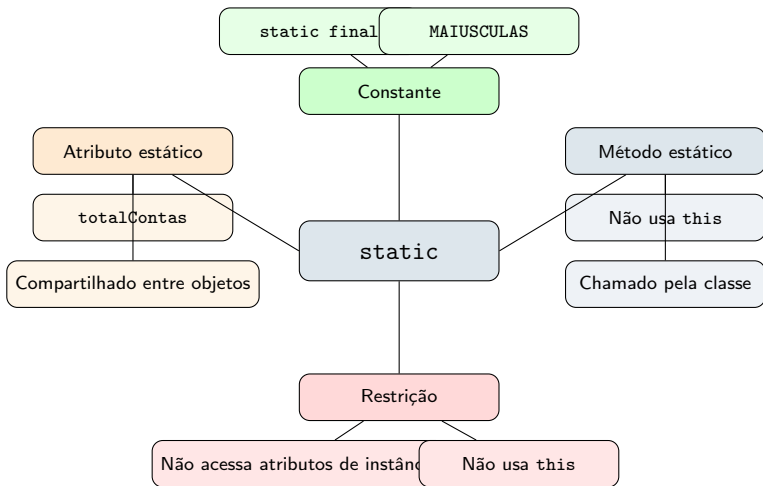
Métodos:

- static boolean
entrar(String placa)
- static void sair()
- static int
getVagasLivres()
- static boolean
estaLotado()

</> Teste esperado

```
1 System.out.println(  
2     Estacionamento.getVagasLivres()); // 50  
3 Estacionamento.entrar("ABC-1234");  
4 Estacionamento.entrar("DEF-5678");  
5 System.out.println(  
6     Estacionamento.getVagasLivres()); // 48  
7 Estacionamento.sair();  
8 System.out.println(  
9     Estacionamento.estaLotado()); // false
```

Mapa mental: static em Java



Checklist: antes de usar static

✓ Devo responder SIM

- ☐ O valor é o mesmo para todos os objetos?
- ☐ O método não usa `this`?
- ☐ O método não acessa atributos de instância?
- ☐ Faz sentido chamar sem criar um objeto?

✗ Se responder SIM a estes, NÃO use

- ☐ O valor varia de objeto para objeto?
- ☐ O método usa `this.algo`?
- ☐ Estou usando `static` só para parar o erro?
- ☐ Estou substituindo orientação a objetos?

💡 Regra de ouro

Use `static` com critério. O abuso de membros estáticos transforma código orientado a objetos em código procedural disfarçado.



Conceitos-chave

- `static` → pertence à classe
- Uma única cópia na memória
- Acesso pelo nome da classe
- `static final` → constante
- Método `static` não usa `this`



Exemplos reais

- `Math.sqrt()`, `Math.PI`
- `Integer.parseInt()`
- Contadores de objetos
- Constantes de limite
- Classes utilitárias

“Instância para o que varia por objeto. Estático para o que é da classe”